PROTEUS: A MODULAR FRAMEWORK FOR AGENT SELF-IMPROVEMENT

The Proteus Authors

ABSTRACT

Research on agent self-improvement spans prompt search, context accumulation, harness-code optimization, and persistent multi-agent discovery. Comparing and extending these methods requires more than a common model endpoint: candidate representations, execution roles, feedback access, and recovery semantics must remain explicit. We present PROTEUS, a unified framework that separates method logic and typed stages from immutable candidate artifacts, role-specific harnesses, and external evaluation. An optimizer and a task subject can use independently configured harnesses and models, subject to declared capabilities and artifact protocols. Implemented profiles cover GEPA, ACE, Meta-Harness, Combee, CORAL, and Native evolution; reusable components support ACE–Combee and GEPA–Combee compositions and explicit self-evaluation stages. The Native reference method separates goals from evaluator evidence; its common profile activates viable candidates without an outer score filter and retains nonviable candidate edits for repair. Shared services preserve lineage, evidence visibility, and recovery state. Native sweeps have an interactive viewer, while generic recipes provide JSON/CSV exports and study summaries. We document fourteen historical adapted method profiles and one Native mechanism profile across sixteen recipes, sixteen bounded live harness-role pilot cells, and source-bound engineering qualification. These establish implementation coverage and inspectability, not matched-paper reproduction, unrestricted interchangeability, or statistically demonstrated performance gains. PROTEUS provides an extensible experimental substrate for studying self-improvement without requiring every method to adopt one evolution loop.

1 INTRODUCTION

Agent self-improvement changes the system around a model as well as the instructions sent to it. GEPA searches prompt components using reflective feedback (Agrawal et al., 2026; GEPA contributors, 2026); ACE accumulates a structured playbook (Zhang et al., 2026; ACE contributors, 2026); Meta-Harness searches executable memory and orchestration code (Lee et al., 2026; Meta-Harness contributors, 2026); Combee reorganizes prompt learning through parallel aggregation (Li et al., 2026; GEPA contributors, 2026); and CORAL coordinates persistent discovery agents (Qu et al., 2026; CORAL contributors, 2026). These approaches expose different experimental objects, update schedules, and forms of evidence. Studying them in a common system therefore requires preserving their algorithmic distinctions while making their execution and records compatible.

A reproducible self-improvement experiment is larger than an optimizer function. The researcher must identify the exact candidate that ran, the harness and model that proposed it, the subject that executed it, and the evaluator that assigned its score. An interrupted call may have incurred cost without returning a usable result. A hidden test may be recorded for the owner but must remain unavailable to the next proposal. A source edit may exist on disk without ever becoming the executing agent. If these distinctions are implicit, an apparent improvement can instead reflect a changed runtime, leaked feedback, a different selection rule, or missing work.

PROTEUS makes these experimental choices separate, versioned contracts. A *method* owns the optimization loop; *stages* implement typed transformations; *candidates* identify immutable artifacts; and *harness roles* bind model-backed work to execution adapters. Subjects run candidates on tasks, while evaluators produce formal measurements and control their disclosure. Search policies, parallel

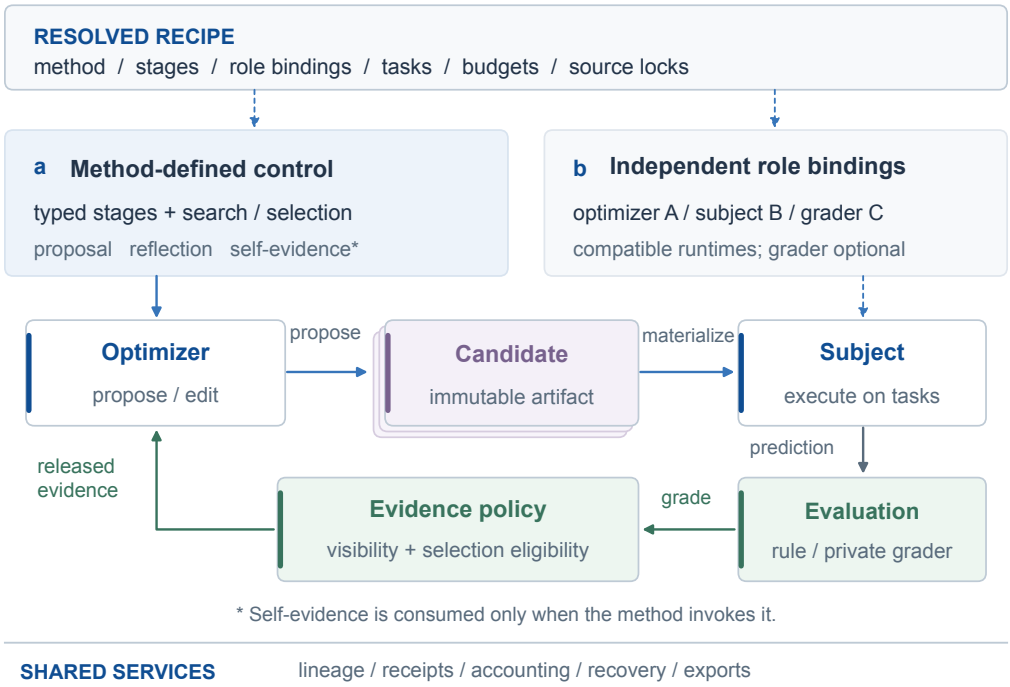


Figure 1: A common experimental substrate, with method-specific control. A resolved recipe binds method-defined control and independent harness roles. The central path separates proposal, immutable candidate materialization, and task execution. Formal evaluation returns evidence only through the declared visibility and selection policy. Typed stages, including optional self-evidence, are invoked by the method; the depicted services do not prescribe one universal optimization loop. The lower band preserves common execution records.

workflows, persistent workers, accounting, and recovery are shared services rather than mandatory steps in one universal algorithm. Figure 1 summarizes the boundaries.

Harness agnosticism means that a method does not select an adapter through a hard-coded method–harness pair. A recipe can bind a coding proposer and a task subject to the same integration or to different compatible integrations, and can choose their models separately. This is an interface property with concrete limits: a playbook, a prompt overlay, and a source tree still need the correct subject protocol. A supported adapter name does not make every method, role, session mode, or artifact combination valid. Preflight checks and an explicit evidence matrix define the supported subset.

The report makes three contributions. First, it specifies a common decomposition of method, typed stage, artifact, harness role, and evaluator, with concrete ACE–Combee and GEPA–Combee compositions. Second, it presents Native as a reference self-evolution method: goals and external scores remain separate, the harness can revise its own tests, and the common profile advances viable candidates without an outer score filter while preserving completed but nonviable candidate edits for repair. Third, it presents a traceable inventory separating historical adapted studies, bounded live role pilots, and source-bound engineering qualification. Existing evolution toolkits already provide extensible infrastructure; the contribution here is the implemented framework and specified Native protocol, not priority for a unified RSI system or a proven quality advantage. Figure 2 details the Native state transition.

The evidence supports a research toolkit whose available recipes and failure boundaries can be inspected. Historical AIME and Formula runs contain complete declared task splits, while later pilots exercise specific Pi, DSH, and Codex roles. Current preparation checks cover installed extensions, common-trace composition, candidate activation, and a directed CORAL recovery protocol. Their sources and experimental meanings differ, so we do not pool them into a single benchmark or release

Table 1: Source-audited system coverage as of November 2, 2026. ✓: explicit implementation or documented protocol; ✗: the named method is not shipped in the inspected built-in registry/library; ?: the specified shared contract was not established by this audit. A cross does not mean inexpressibility, and a check does not mean matched-paper reproduction. The lower block covers the five method families studied here, not the full method catalog of each system. See Appendix E for scope and pinned sources.

Interface or integration	Proteus	Penguin
Shared infrastructure and explicit contracts		
Multiple improvement methods	✓	✓
Versioned candidate resources	✓	✓
Independent proposer / task models	✓	✓
Private evaluation / held-out protocol	✓	✓
Harbor / Docker execution path	✓	✓
Typed reusable stage / workflow service	✓	?
Separate visibility / selection-eligibility fields	✓	?
Durable dispatch reservation / reconciliation	✓	?
Built-in adaptations of this report's method families		
GEPA	✓	✗
ACE	✓	✓
Meta-Harness	✓	✗
Combee	✓	✗
CORAL	✓	✗

gate. Generic builders continue to require new model-result qualification. The intended use is to run, inspect, and extend explicitly specified experiments, with missing evidence remaining visible.

2 Scope and Related Work

Experimental scope. We study changes to prompts, playbooks, memory, skills, tools, and harness code at fixed model weights. A *harness* is the execution system surrounding a model, including its tools, context handling, and process or session behavior. A *candidate artifact* is the mutable object under study; it need not be the optimizer's harness. We use *self-improvement* for methods that produce and evaluate such changes, without presuming that every trajectory improves. Recursive self-improvement is a research direction that the framework can help investigate. Sustained recursive returns, autonomous expansion of capability, and model training are not demonstrated in this report.

Prompt, context, and program adaptation. The five integrated families provide complementary mechanisms rather than interchangeable implementations of one algorithm. GEPA's reflective search and ACE's incremental playbook updates motivate separate search and structured-artifact interfaces. Combee adds an aggregation mechanism usable with more than one base learner. Meta-Harness motivates code artifacts and history-rich proposal workspaces, whereas CORAL motivates independent workers, shared knowledge, and event-triggered interventions. We reuse their published ideas and pinned official implementations where specified, cite their original papers at introduction, and record official software and source revisions in Appendix A. A Proteus integration profile may cover only a subset of an upstream method's options.

Evolution toolkits. OpenEvolve provides program-evolution infrastructure (OpenEvolve contributors, 2026), while Penguin Harness supplies multiple RSI toolkits over versioned agent state and shared evaluation records (PrismShadow Team, 2026). Candidate search, archives, and extensible improvement workflows are established building blocks. Table 1 records a bounded source audit of infrastructure and selected method coverage. Proteus's design is described directly in Section 3; the table does not establish overall breadth or performance superiority.

A Native method within an extensible framework. Proteus pairs the research toolkit with a reference self-evolution method: the executing harness inspects and changes its own declared state, while the protocol keeps the goal, external measurements, and activation rule separate. Its common-contract profile uses viability-gated continuation rather than an outer best-score filter. Pi, DSH, and Codex supply execution integrations rather than competing optimization methods (Zechner &

Pi contributors; DeepSeek, 2026; OpenAI, 2026). Section 4.1 specifies the update rule and the distinction between its legacy and common profiles.

Research toolkits and execution backends. The organizational model resembles recipe-oriented research toolkits such as ESPnet (ESPnet contributors, 2026): expose established methods, common interfaces, and reproducible configurations, while allowing new components to enter at defined extension points. The analogy concerns organization, not comparable maturity or task coverage. Harbor supplies agent evaluation and container infrastructure (Harbor contributors, 2026); Proteus uses pinned Harbor/Docker execution where declared, while retaining ownership of method state, candidate identity, feedback policy, and experiment aggregation. Container execution and optimization logic remain distinct responsibilities.

3 Framework Design

A resolved recipe specifies the experimental contract before execution. Write this abstractly as

$$\mathcal{R} = (M, \mathcal{S}, \mathcal{A}, \mathcal{H}, \mathcal{T}, \mathcal{E}, \mathcal{V}, \mathcal{B}, \mathcal{L}), \tag{1}$$

where M is the method, $\mathcal{S}$ its configured stages, $\mathcal{A}$ the artifact protocol, $\mathcal{H}$ role bindings, $\mathcal{T}$ tasks and splits, $\mathcal{E}$ evaluators, $\mathcal{V}$ evidence policy, $\mathcal{B}$ supported budgets, and $\mathcal{L}$ source and dependency locks. This notation summarizes the implementation's separate specifications; it is not a new optimization objective. Locks and an immutable run manifest make changed experimental conditions distinguishable from continuation of an existing run.

3.1 Methods, stages, and search policies

Method plugins own proposal scheduling, acceptance, stopping, final selection, and private algorithm state. They call framework services through `Method.run(services, restored_state)` and checkpoint the state needed to continue their own loop. Consequently, an ACE sample update does not have to become a Native episode, and a CORAL worker does not have to become a synchronous parent-mutation step. The shared service boundary makes operations comparable at the record level without forcing unlike algorithms into an artificial common schedule.

Stages provide typed inputs and outputs for reusable work such as reflection, code proposal, or self-assessment. A stage obtains model-backed behavior through its configured actor rather than creating an unrecorded provider call. Sequential, conditional, finite-loop, map, and workflow composition support the implemented profiles. Adding a stage declaration alone does not insert it into every method: the method must expose or invoke that stage and specify how its output affects later actions. A new update rule or control loop therefore requires a plugin when no existing configurable mechanism expresses it.

Search services separate the generation of candidates from decisions about their retention and reuse. An archive is an inspectable set of candidate records and eligible measurements. Parent selection chooses which archived candidates condition a proposal; archive policy decides what to retain; final selection determines the output candidate. These decisions receive a permitted score view and a supplied random-number generator, and return candidate identifiers with reasons. Scalar, uniform, and Pareto-style policies are useful components, but are not aliases for a full upstream algorithm. The GEPA profile additionally retains its pinned instance-frontier and parent-sampling semantics.

3.2 Candidate identity and harness roles

Candidates are immutable, content-addressed artifacts with explicit parent edges. The artifact store supports text components, structured records, and file trees, together with checked deltas and materialization. A candidate identifier refers to the object that was proposed; its lineage records the update's origin. Running that candidate additionally requires an execution configuration. The system therefore records both artifact identity and effective runtime identity, including a pinned base and projection when a prompt component is overlaid on a source tree. Deleting a declared component removes the corresponding base path instead of silently restoring old content.

A `HarnessIntegration` binds an adapter factory, source identity, options, and declared capabilities to versioned role plugins. Ordinary actors, coding workspaces, and Native evolution roles use the

roles that the integration advertises. Each recipe independently supplies an `ActorSpec`, including model and runtime options, for the optimizer or proposer, subject, and optional grader. The factory can know the role without knowing the optimization method. This supports both same-harness experiments and compatible cross-harness bindings without maintaining a separate registry for every method–harness pair.

Role independence does not erase representation or lifecycle requirements. The common candidate subject runs a file tree, or an explicitly declared projection, through its configured actor and reads the task prediction from the specified output path. A GEPA text-component candidate and an ACE playbook require their own compatible projections or runners. Source validators check the exact candidate before execution; missing output, invalid source, and infrastructure failure remain distinct outcomes. The common candidate bridge currently requires independent serial target trials, and its materialization layer is trusted host execution rather than a process sandbox. Container adapters supply only their declared additional boundary.

3.3 External evaluation and evidence visibility

Evaluation consumes a task, a recorded subject prediction, and any private reference through a declared evaluator. Deterministic exact-match or domain-specific graders share this interface with a managed model-judge protocol. In the latter, a separate grading actor receives private input and produces an identity-bound result with its own usage record. The implemented managed grading context permits one dispatch; a multi-call judge needs an explicitly bounded protocol. A malformed grading response is an infrastructure failure with an unavailable score, not a negative task outcome fabricated to complete a table.

Feedback visibility and selection eligibility are independent experimental choices. A visible score can inform learning where the method permits it; a hidden reference, grader request, or final score remains outside optimizer inputs. A measurement can also be recorded without being eligible for parent or final selection. Owner-side private export is separate from what an agent sees. These boundaries allow several evaluators to coexist and permit validation selection followed by hidden testing without treating all recorded numbers as optimization feedback. They govern supplied evidence; trusted Python plugins still require appropriate execution isolation if adversarial behavior is in scope.

A self-evaluation stage produces evidence whose author and consumption rule must be explicit. It can inspect allowed traces, critique a candidate, or propose additional checks, provided the chosen method invokes it. Its output does not automatically replace the independent external score or become a hidden-test oracle. The framework can preserve both channels, enabling later experiments on whether self-evidence is useful, calibrated, or misleading. Current extension examples demonstrate this separation mechanically; they do not establish the quality of self-authored evaluators.

3.4 Execution, accounting, and recovery

Execution services distinguish a trial, an explicit sequence with carried state, and a persistent worker lifecycle. Pinned Harbor/Docker profiles can provide candidate loading, independent verification, source builds, and cleanup evidence. A persistent worker additionally needs a durable identity, its own workspace, a session policy, and a way to submit immutable candidates for formal evaluation. Shared knowledge uses versioned, sourced records and compare-and-swap publication; event policies can request reflection, consolidation, or a change of direction. The existence of a shared snapshot is not proof that another worker consumed it.

The managed ledger reserves resources before dispatch and settles observed usage against the corresponding operation. Optimizer, subject, grader, build, retry, and worker activity retain attributable records within their supported boundaries. A reservation is not an invoice. Unknown tokens, provider-call counts, monetary charges, or elapsed intervals remain unknown; unsupported finite limits fail preflight. A terminal operation with incomplete usage differs from an operation whose external completion is unknown. In the latter case, the unresolved reservation and dispatch state prevent an ordinary resume from silently issuing the work again.

Recovery restores method checkpoints and committed operation results under the original source and configuration locks. Backend-specific receipts can reconcile independently completed work, including the supported Harbor worker path. Without trustworthy completion evidence, the system

stops for reconciliation instead of inferring that a timeout means no work occurred. Candidate failure also remains inspectable: A completed Native staged-source episode preserves a nonviable candidate for repair while subsequent execution retains the last valid source. Its activation proof binds the next executing source or binary to the validated artifact. These guarantees concern supported state transitions and receipts, not universal recovery from every provider or worker failure.

3.5 Inspection and researcher interfaces

Inspection has two complementary surfaces. The local Native viewer exposes Evolution, Measurements, Configuration, and Snapshots for sweep records. It distinguishes failed candidates from active checkpoints and preserves missing metrics. Generic recipes instead export JSON and CSV records for tasks, candidates, lineage, operations, usage, and timing; study specifications aggregate explicitly declared conditions and seeds. The Native viewer does not currently render every generic recipe. Descriptive surface or trajectory distances are available for supported artifacts, but are not universal quality measures across prompts, memory, and source code.

The versioned `proteus.api.v1` surface exposes contracts and services for methods, stages, subjects, evaluators, actors, artifacts, and search policies. Release preparation freezes 100 public exports and 64 contract schema identities. External templates cover method, stage, evaluator, subject, and policy extensions and have installed-distribution checks outside the checkout. Private SQL tables, journal layout, and arbitrary internal service fields are not made stable merely by importing a public type. A new contribution must declare its capabilities, source identity, feedback use, and recovery behavior alongside its algorithm.

4 Native Self-Evolution and Composable Recipes

4.1 Native: goal-aware, viability-gated self-evolution

Native studies self-evolution when the objective is not necessarily identical to an external score. A natural-language goal can be fully operationalized by a benchmark, or only partly measured by it. The default protocol asks the harness to judge that relationship and to add or revise its own tests when useful. It does not mandate extra tests or treat a self-reported score as an independent measurement. The legacy interface also permits a goal without an evaluator, or open-ended exploration without a supplied goal; those are explicit settings rather than evidence of autonomous capability growth.

The reference episode uses *observe, propose, act, reflect*. The active harness inspects its state and permitted evidence, chooses a scoped change, edits its pending candidate, and records outcomes and unresolved risks. The four phase names alone are not the contribution. The relevant experimental choices are which goal and evidence are supplied, what state the harness can change, which implementation executes each phase, and how a candidate becomes active. For supported staged-source adapters, all phases run the frozen last-valid source, including reflection; pending edits take effect only after validation and activation. Self-authored tests remain part of the harness state, distinct from the external grader.

The common-contract profile separates the active harness A_t from the working candidate W_t. For an episode that completes successfully and publishes a candidate, let it produce $\widetilde{W}_{t+1} = F_\theta(A_t, W_t, G, V_t)$, where G is the supplied goal, V_t is permitted evidence, and the model weights θ remain fixed. With a model-free viability check v, its transition is

$$W_{t+1} = \widetilde{W}_{t+1}, \qquad A_{t+1} = \begin{cases} \widetilde{W}_{t+1}, & v(\widetilde{W}_{t+1}) = 1, \\ A_t, & v(\widetilde{W}_{t+1}) = 0. \end{cases} \tag{2}$$

There is no condition requiring an external score to increase. A viable candidate becomes active even if its measured task score decreases; an invalid candidate remains the exact working state for a later repair episode while execution retains A_t. An actor/stage failure or an uncertain external operation stops this transition and follows the shared failure/recovery protocol; it is not silently treated as a viability rejection. This defines a continuation policy, not a monotonic-improvement guarantee. The final candidate is the last valid one, not an outer-loop score maximum.

The common profile requires observed evaluation pools, evaluates the initial and each viable candidate, and runs optional hidden tests after evolution. It binds evolution and subject roles to the same harness

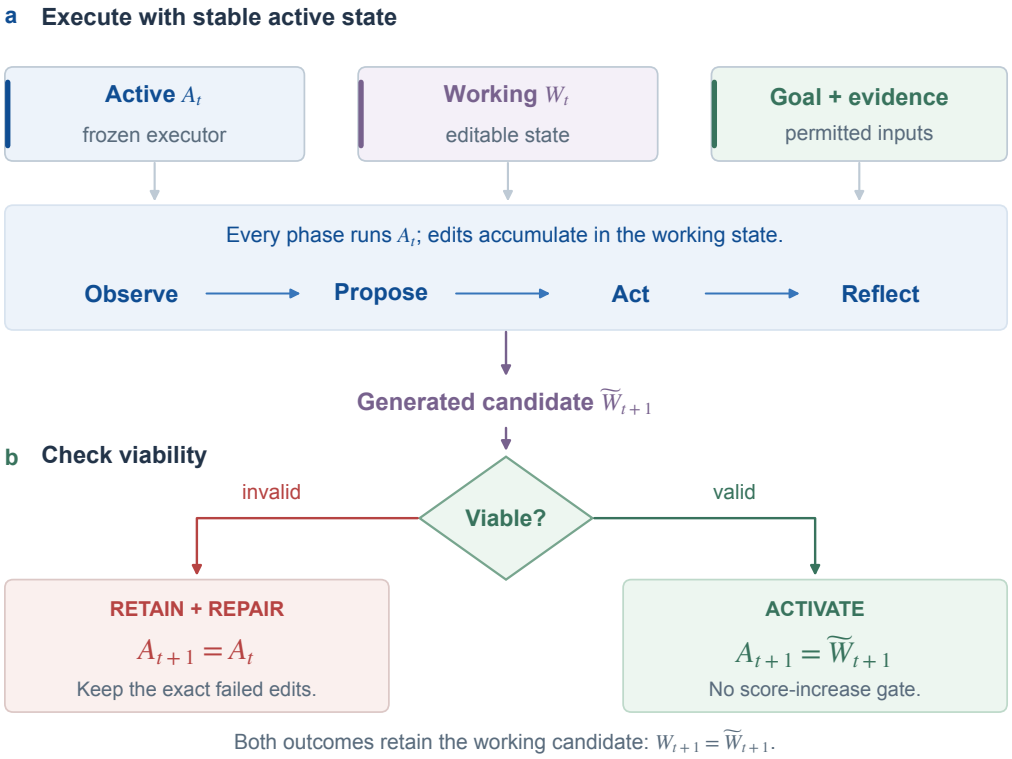


Figure 2: Native common-profile transition. The current active harness executes the episode while changes accumulate in a separate working candidate. A source-bound viability result controls activation. Completed but nonviable edits remain available for repair, and formal scores enter later reasoning only through the declared evidence policy; they do not implement an outer acceptance filter. This figure describes supported staged-source execution, not every legacy adapter's lifecycle.

integration and uses exactly one managed episode stage. Its supported phase controls do not make it an arbitrary stage graph. Legacy Native retains its separate goal/no-goal options, evaluator schedules, dispositions, measurements, and selection settings. Accordingly, the no-outer-score rule in Equation 2 describes the common profile; it must not be generalized to every legacy configuration.

The source-bound Native fixture demonstrates invalid-edit retention, later repair, changed candidate execution, private final grading, and replay. Bounded Pi/DSH pilots additionally exercise one common episode; the broader engineering record tests supported source activation paths. These support the implemented state transition. They do not establish that goal-aware self-evolution beats score-guided search, that self-authored tests are useful, or that useful improvements compound. Such claims require matched comparisons varying objective framing, evidence access, activation/selection policy, and self-evidence while holding tasks, models, initial artifacts, and budgets fixed.

4.2 Reference recipes and researcher workflow

The included recipes demonstrate several ways to reuse infrastructure without equating their algorithms. Table 2 identifies the optimized artifact, controlling mechanism, and empirical boundary of each family. The method documentation and frozen profile configurations specify the actual options; family names alone are insufficient to reconstruct an experiment. In particular, an original-domain builder is an entry point for preparing a new study, not a certificate that the resulting configuration has model-quality evidence.

ACE with Combee. ACE's generator, reflector, and curator operate on a versioned playbook whose structured updates retain stable entries and counters. Combee-ACE reuses that representation and the surrounding evaluation services, while introducing grouped parallel proposal work and aggregation before publication. The implementation distinguishes original samples from copied aggregation

Table 2: Implemented profile boundaries. Reuse is explicit at the artifact, stage, or service layer. A family name is not a claim of all upstream options or matched-paper results.

Family	Candidate	Included mechanism	Evidence boundary
GEPA	Text components	Single-mutation loop, instance frontier, round-robin components	Adapted AIME study; no merge/cache in this profile.
ACE	Structured playbook	Generator, reflector, curator; incremental updates; offline/online	Adapted Formula studies; online stream differs from final held-out scoring.
Meta-Harness	Memory-system code	Historical evidence retrieval, proposal workspace, learning and selection	Historical JSON proposer; actual coding proposers have bounded pilots.
Combee	Base-method artifact	ACE/GEPA aggregation, parallel maps and fixed/dynamic batches	Formula results and separate model-free common trace; no isolated speedup.
CORAL	Program and shared knowledge	Worker generations, formal submissions, sourced knowledge, event policies	Adapted circle packing; separate three-request directed qualification.
Native	Harness source/state	Legacy compatibility; additional common-stage evolution profile	Source activation/repair and external evaluation; no universal Native phase requirement.

occurrences and preserves their provenance. Sequential ACE, naive parallel, fixed Combee, and dynamic Combee therefore form explicit conditions rather than informal choices of thread count. The common frozen-trace check in Section 5.2 isolates these consumption and update mechanisms.

GEPA with Combee. Combee aggregation is also integrated into the pinned GEPA search loop. GEPA retains candidate evaluation, acceptance, component choice, and final testing, while the aggregation operator changes how reflective proposals are formed. This is a second implemented composition, with its own artifact and parser requirements. The existence of both integrations supports component reuse across base learners; it does not establish that a Combee operator can be attached to any method without adapting its input and update contracts. The historical Formula results in this report concern Combee-ACE, not a new Combee-GEPA benchmark.

An explicit self-evaluation experiment. A stage-composing method can insert a self-evaluator between a proposal and the next method decision. The researcher declares its inputs, the permitted evidence view, the output type, and whether that output affects revision, selection, or only logging. An ablation can then remove or replace the stage while keeping tasks, external grading, budgets, and other choices fixed. The tutorial demonstrates such configuration and plugin boundaries. It supplies a way to pose the research question; a useful or calibrated self-evaluator would still require independent empirical validation.

Native evolution as a method. The legacy Native compatibility recipe preserves the existing episode transaction engine and its evaluator schedules and measurements. The additional common Native evolution profile uses ordinary stages, immutable candidates, shared evaluation, and the recovery journal while retaining observe/propose/act/reflect inside its episode stage. It uses neutral disposition and last-valid activation, not outer-score selection. The same integration supplies its evolution and target roles. These paths are documented separately; neither is imposed on ACE, GEPA, Meta-Harness, Combee, or CORAL.

From an experiment idea to a run. The normal workflow is to choose a profile, bind compatible actors and artifacts, declare tasks and evidence policy, then validate and lock sources before running. Replacing an implemented model, prompt, policy, budget, or exposed stage is a configuration change. Introducing a new update rule, actor protocol, or side effect requires the corresponding plugin and its checks. Resume uses the original locks; a changed method or runtime creates a new experiment. Thus, "no model training" is not sufficient for configuration-only reproduction of an arbitrary new method.

Table 3: Historical adapted single-seed results. Counts are correct tasks / evaluated tasks. Offline selected-candidate results and online stream predictions have different meanings; these rows are not a cross-task ranking.

Profile	Initial	Reported outcome	Evaluation meaning
GEPA / AIME	23/30	24/30	Selected, private
ACE / Formula offline	141/200	173/200	Selected, private
ACE / Formula online	141/200	163/200	Pre-learning stream

Agent-facing use and contribution skills. The accompanying documentation commit includes a `proteus-use` skill that guides an agent through profile choice, role binding, validation, execution or saved-run analysis, recovery, and reporting. An optional `proteus-contribute` skill guides extension work. These repository-distributed instructions reference the matching runtime and documentation, distinguish an analysis-only request from a new experiment, and keep evidence and resource limits explicit. They are a usability and contribution interface, not an additional optimizer or a demonstrated performance contribution. Their committed documentation provenance is recorded separately from the source revisions that produced the experimental evidence.

5 Evidence and Supported Conclusions

We evaluate the framework through three distinct evidence strata: historical adapted model studies, bounded live harness-role pilots, and source-bound engineering qualification. The historical inventory contains fourteen adapted method profiles and one Native mechanism profile across sixteen recipes; one profile can contain more than one recipe. These are archived experiments from earlier source revisions, not reruns of the current release-preparation tree. The pilot matrix tests specified role bindings on small tasks. Engineering checks test execution contracts and recovery rather than model quality. Appendix D links each stratum to its source and records what was inspected for this report.

Historical AIME and Formula configurations use the recorded `deepseek-flash` model identifier with temperature zero and thinking disabled. AIME uses an 8,192-token response limit; the Formula roles use 4,096. These are the identifiers and settings in the archived configurations, not assertions of equivalence to an upstream paper's model or to an unreported provider-resolved revision. Model seeds do not establish deterministic remote inference. We retain baseline, selected, and last-candidate distinctions, report task counts directly, and avoid pooled cross-task rankings or single-seed significance claims.

5.1 Historical task results: examples of supported study protocols

The AIME and standalone Formula studies illustrate why a shared record format must retain evaluation semantics. GEPA/AIME uses 45 training, 45 validation, and 30 private tasks; its recorded baseline solves 23 private tasks and its selected candidate solves 24. ACE/Formula uses 500/300/200 tasks. Offline ACE records 141/200 for the initial playbook and 173/200 for the selected playbook, while online ACE records 163/200 pre-learning stream predictions against a 141/200 baseline. The online number describes predictions made before each learning window, not an untouched test of the final playbook. Table 3 keeps these quantities separate.

Offline ACE also shows why selecting the last artifact would change the reported experiment. The retained candidate is selected after 200 training examples, whereas the last playbook follows all 500. Only the initial and selected playbooks have private measurements; the last playbook's private score is unavailable. Proteus records the selected candidate, the final execution artifact, their hashes, and the visible-only selection rule separately. The 16-percentage-point observed difference is a result of this one adapted trajectory, not a claim about expected gain or the last candidate.

The historical Meta-Harness-inspired study covers USPTO, Symptom2Disease, and LawBench with NoMemory and FewShotAll baselines plus three generated memory programs. Its proposer makes one JSON call that returns three programs, rather than conducting the original paper's extended tool-using coding search. Each system has 450 learning callbacks and 260 visible predictions over 130 distinct visible tasks. Four selected systems receive 412 private tasks each. Table 4 reports their per-domain counts; the generated systems do not dominate FewShotAll. This experiment

Table 4: Historical Meta-Harness-inspired MCE private results. The proposer is one JSON call producing three programs; four systems were selected for private evaluation. The unselected lesson-distilled program has no private result. Counts are correct / tasks, and generated systems do not dominate FewShotAll.

System	Origin	USPTO	Symptom2Disease	LawBench
NoMemory	Baseline	16/100	151/212	28/100
FewShotAll	Baseline	29/100	183/212	55/100
Contrastive boundary	Generated	29/100	162/212	45/100
Dual-pool retrieval	Generated	30/100	152/212	52/100

Table 5: Historical online Combee/Formula conditions, seed 0. Accuracy is pre-learning stream correct / 200. API upper cost is USD, not an invoice or canonical native cost. Elapsed hours follow the recorded study timer; shared resources and continuations preclude an isolated speedup claim.

Condition	Initial	Stream	API upper (USD)	Study elapsed (h)
ACE	139/200	160/200	1.738	4.62
Naive	137/200	157/200	0.705	2.81
Fixed	140/200	159/200	0.772	3.00
Dynamic	137/200	162/200	0.742	2.89

demonstrates a code-artifact and learning/evaluation protocol under its recorded adaptation, while current coding-proposer support is assessed separately by pilots.

The historical CORAL-inspired circle-packing arms compare one proposer making 20 submissions with two proposers making ten each, using one evaluation worker. The recorded selected normalized scores are 0.928722 and 0.900208, respectively, from an initial 0.364102. Twelve versus eleven submissions succeed; eight versus nine time out. Final confirmation repeats the same public 26-circle problem. These results do not establish a multiworker advantage, hidden-task generalization, or the behavior of the original persistent native agents. The locally unrecovered historical cross-worker chains are not repaired by relabeling a later protocol check.

5.2 Composition: historical measurements and a controlled trace

The Combee/Formula inventory contains sequential ACE, naive parallel, fixed Combee, and dynamic Combee in both offline and online modes. The four online conditions have accepted pre-learning accuracy, independently recorded API upper-cost estimates, and study elapsed measurements (Table 5). The API amounts are not invoices and do not overwrite unknown canonical native USD totals. Study elapsed time includes the recorded study boundary and differs from the sum of overlapping operation durations. Shared host resources and continuation histories prevent interpreting the table as an isolated speedup experiment; the different baseline scores also remain visible.

A separate model-free check feeds eleven frozen original task/prediction/feedback/reflection records to the four production ACE variants. All arms use the same H0 and seed. Sequential ACE curates each example; naive parallel uses batches 3/3/3/2; fixed Combee uses the same batch schedule with copied groups and reduction; dynamic Combee uses 1/2/4/4 under a controlled clock. Curator invocation counts are 11, 4, 12, and 12, respectively, while every arm makes 22 generator invocations. Provider-call count is zero. The auditor binds actual payload bytes, source/configuration locks, grouping, requests, updates, and lineage. This establishes distinct, reusable composition semantics under common inputs; synthetic timing and deterministic responses cannot establish learned quality or model speedup.

5.3 Harness-role interoperability: sixteen bounded live cells

The live pilot inventory contains sixteen accepted cells across Pi, DSH, and Codex (Table 6). Native and GEPA cells use actual Pi/DSH source adapters as task subjects; GEPA's reflector is a separately managed chat actor. Other cells put the coding harness in the ACE/Combee generator role, Meta-Harness proposer role, or a bounded CORAL worker wrapper. The tasks are deliberately small and their role placement is part of the evidence. A checkmark therefore means that the named minimum cycle was observed, not that all roles of a complete benchmark ran on that harness.

Table 6: Sixteen accepted bounded live cells. ✓ = the listed minimum cycle has source-bound live evidence; — = not established by this pilot inventory. The harness role differs by row. These are integration pilots, not full benchmarks or comparisons of model quality.

Method/profile	Pi	DSH	Codex	Role exercised by the named harness
Native common	✓	✓	—	Evolution actor and candidate subject; one episode.
GEPA	✓	✓	—	Candidate subject; separate managed chat reflection.
ACE	✓	✓	✓	Generator executing the current playbook.
Combee-ACE	✓	✓	✓	Independent generator workspaces; one map/reduce batch.
Meta-Harness	✓	✓	✓	Coding proposer; generated memory-system programs execute.
CORAL bounded	✓	✓	✓	One supervised worker; two fresh coding workspaces.
Combee-GEPA	—	—	—	Implemented composition, outside this live matrix.

Successful cycles include outcomes that do not improve on their baselines. The Pi ACE and Combee-ACE pilots modify and execute a playbook but retain the initial candidate, with private scores unchanged at zero. Meta-Harness coding proposers generate executable systems, yet the selected baselines tie or exceed them on the observed pilot tasks; those generated systems need not have private scores. Other cells demonstrate a changed policy or source artifact, independent scoring, and zero additional dispatches on completed resume. These outcomes support interoperability and auditability while resisting an artificial all-positive performance story.

The matrix leaves Codex Native/GEPA and all Combee-GEPA live role cells unclaimed. The additional CORAL role pilots use one supervised worker invoking fresh coding workspaces, not persistent native-session multiworker execution. Requested/applied model identifiers are preserved separately from missing provider-resolved identities. Cell-level evidence spans several recorded commits and includes corrected failed attempts. It is appropriate for the bounded integration claim, but cannot be assembled into a single-source release gate or a controlled comparison of harness quality.

5.4 Current engineering and directed protocol qualification

A directed CORAL qualification specifically tests shared-content delivery, intervention, and controller-crash recovery. The accepted live run contains exactly three completed paid provider requests, reporting 1,141 input tokens and 113 output tokens; USD cost is unknown. Readback follows the actual delivered content, applied intervention, original request identities, and settled receipts. Resume performs the declared final confirmation evaluation without an additional model request. This closes a narrow protocol chain that historical knowledge snapshots alone could not prove. It does not establish worker-crash recovery, native model-session continuation, simultaneous provider dispatch, or a benefit from collaboration.

The complete preparation smoke, run 41398194611, passes Offline, Minimal, LLM, Pi, DSH, and Codex jobs in one source-bound run. The offline job reports 1,842 passing tests and 91 skips; the live jobs include the declared source-edit, activation, and rejection checks. This is engineering qualification. Pi/DSH diagnostic archives are bounded and sanitized: Pi's 20 phase outcomes and 20 of DSH's 24 outcomes return 137 under controller caps, with incomplete usage streams. Codex's original strict-checker log establishes its checked edit/binary behavior, but its raw trace, snapshot, and binary archive were not exported. Passing this bounded gate neither proves complete billing nor makes the retained artifacts a full replay archive.

Independent generic readback reconstructs nine report/CSV files for each of five saved runs, covering 92 task rows and 30 candidates with no unreported rollout. Costs and timing intervals that were missing remain missing. Installation evidence covers 22 checks outside the checkout, including extension templates, exports, and study comparison. These results support the usability of the records and public extension boundary. They do not measure researcher productivity, and the current report does not rerun models, containers, or benchmarks to reproduce them.

Taken together, the evidence shows implemented algorithm profiles, concrete role-level execution, and inspectable state transitions under declared conditions. Historical task improvements illustrate that the substrate can carry substantive studies; negative pilot outcomes and retained failures show why implementation acceptance is a separate criterion. Controlled trace checks expose reusable

mechanics, and directed qualification tests a specific recovery chain. None of these alone establishes a general self-improvement advantage, a universal harness interface, or a successful future release tag.

6 Limitations and Research Directions

The principal empirical limitation is the absence of matched, multi-seed comparisons across the implemented methods and harness choices. Historical profiles use adapted models, different algorithm settings, source revisions, and task protocols. The bounded coding-proposer pilots do not replace full coding-search benchmarks. To estimate an effect of a framework component, a study must hold its task splits, models, starting artifacts, feedback, selection, and enforceable budgets fixed while varying that component. The present evidence does not estimate population-level gains, state-of-the-art performance, or the causal contribution of composition, self-evaluation, or shared knowledge.

Native's viability gate establishes executability, not useful improvement. A runnable regression can become the active candidate, and self-authored tests can be incomplete or misleading. Matched objective-framing, score-selection, and self-evidence ablations are needed before claiming a quality benefit.

The framework's modularity is limited by implemented artifact and execution contracts. Some new research ideas need a new stage or policy; others require a method, subject, evaluator, runtime, or service extension. Session continuity, concurrency, and isolation are capabilities rather than freely interchangeable options. Plugins executing in the controller are trusted, and the common subject materializer is not itself a sandbox. A richer support matrix requires actual contract and execution evidence for the new combinations, not additional configuration labels. Native compositions with ACE, GEPA, Combee, or CORAL are reserved for a later version; this report covers ACE–Combee and GEPA–Combee.

Accounting and inspection are constrained by what providers and harnesses expose. Unknown provider-call totals or monetary charges remain unknown, and upper-cost estimates retain their separate meaning. Completed-resume checks cannot establish recovery from every uncertain external effect. The directed CORAL result covers a controller interruption, not a worker or native-session crash. Bounded Pi/DSH diagnostics and the Codex checker-only archive limit independent reconstruction of full native conversations and binaries. Generic JSON/CSV reports and the Native interactive viewer also have different current coverage.

The coverage table is a source audit, not a controlled evaluation. Its selected method rows do not establish overall breadth or maturity. Integration-effort studies should measure the changes needed to add a stage or method under fixed task and runtime conditions; interruption studies should test retained evidence and continuation decisions. Until then, configurability, reliability, and research productivity remain implemented mechanisms and research hypotheses rather than measured comparative advantages.

The next research steps are matched harness-role ablations, full coding-proposer studies, multi-seed method comparisons, and controlled tests of self-evidence and cross-worker knowledge use. A particularly useful design would compare the same candidate update under different proposer/subject bindings while preserving the evaluator and allowed evidence. Long-horizon experiments are additionally needed before making claims about sustained recursive improvement. These are open research questions; their absence narrows the conclusions of this report rather than invalidating the supported infrastructure claims.

7 Conclusion

PROTEUS combines a modular experiment framework with the Native self-evolution reference method. The framework separates methods, artifacts, harness roles, and evaluation; Native specifies goal/evidence use, viable-candidate activation, and nonviable-edit repair. Implemented compositions, historical studies, bounded pilots, and engineering checks establish inspectable execution within declared limits, not a general quality advantage. The interface records what changed, what executed, and what remains unknown, enabling controlled investigation of self-improvement methods.

REFERENCES

ACE contributors. ACE: Official implementation. Software repository, 2026. URL https://github.com/ace-agent/ace. Exact integration pin is listed in Appendix A.

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning, 2026. URL https://arxiv.org/abs/2507.19457.

CORAL contributors. CORAL: Official implementation. Software repository, 2026. URL https://github.com/Human-Agent-Society/CORAL. Exact integration pin is listed in Appendix A.

DeepSeek. DeepSeek Harness. Official software repository, 2026. URL https://github.com/deepseek-ai/deepseek-harness. Source used in role pilots: 99f6f02fecdb7dff40c3fbc9470f5907c29f74ca.

ESPnet contributors. ESPnet: End-to-end speech processing toolkit. Official software repository, 2026. URL https://github.com/espnet/espnet. Accessed November 2, 2026.

GEPA contributors. GEPA: Official implementation. Software repository, 2026. URL https://github.com/gepa-ai/gepa. Exact integration pins are listed in Appendix A.

Harbor contributors. Harbor: Framework for evaluating and improving agents. Official software repository, 2026. URL https://github.com/harbor-framework/harbor. Accessed November 2, 2026.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026. URL https://arxiv.org/abs/2603.28052.

Hanchen Li, Runyuan He, Qizheng Zhang, Changxiu Ji, Qiuyang Mang, Xiaokun Chen, Lakshya A Agrawal, Wei-Liang Liao, Eric Yang, Alvin Cheung, James Zou, Kunle Olukotun, Ion Stoica, and Joseph E. Gonzalez. Combee: Scaling prompt learning for self-improving language model agents, 2026. URL https://arxiv.org/abs/2604.04247.

Meta-Harness contributors. Meta-Harness: Official implementation. Software repository, 2026. URL https://github.com/stanford-iris-lab/meta-harness. Exact integration pin is listed in Appendix A.

OpenAI. Codex. Official software repository, 2026. URL https://github.com/openai/codex. Individual pilot and smoke records retain their actual source and binary bindings.

OpenEvolve contributors. OpenEvolve: Open-source implementation of AlphaEvolve. Official software repository, 2026. URL https://github.com/algorithmicsuperintelligence/openevolve. Accessed November 2, 2026.

PrismShadow Team. PenguinHarness: Unified and stable RSI framework. Official repository and technical documentation, 2026. URL https://github.com/Prism-Shadow/penguin-harness/tree/7406d13e2f4b3ba0fe495feed32c172f3484be3b. Core package version 0.2.18. Source audit, November 2, 2026. Commit 7406d13.

Ao Qu, Han Zheng, Zijian Zhou, Yihao Yan, Yihong Tang, Shao Yong Ong, Fenglu Hong, Kaichen Zhou, Chonghe Jiang, Minwei Kong, Jiacheng Zhu, Xuan Jiang, Sirui Li, Cathy Wu, Bryan Kian Hsiang Low, Jinhua Zhao, and Paul Pu Liang. CORAL: Towards autonomous multi-agent evolution for open-ended discovery, 2026. URL https://arxiv.org/abs/2604.01658.

Mario Zechner and Pi contributors. Pi Agent Harness. URL https://github.com/earendil-works/pi/tree/d981de1229ef899957bbe968bc8dcda02a21f477. MIT-licensed software; source revision d981de1229ef899957bbe968bc8dcda02a21f477.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models, 2026. URL https://arxiv.org/abs/2510.04618.

A Source Pins and Attribution

Each method is cited by its original paper and official code. The paper version identifies the reference text; the commit identifies the reused implementation. Neither identity establishes that a Proteus experiment matches all original model, budget, split, or runtime choices. Full adaptation notes are in the source-pinned method reference record. Table 7 lists the software pins used by the integrations.

Table 7: Original papers and official implementation pins. Linked commits contain full revision identities. Combee aggregation and dynamic sampling use different source revisions.

Method	Paper version	Official software / pinned code
GEPA	2507.19457v2	gepa-ai/gepa: 8b0ce6cd99
ACE	2510.04618v3	ace-agent/ace: 82709de050
Meta-Harness	2603.28052v1	stanford-iris-lab/meta-harness: 8123ccabe2
Combee	2604.04247v2	gepa-ai/gepa aggregation: f1cfdee285; dynamic sampler proposal: f9e2d43081; Formula also uses ACE above.
CORAL	2604.01658v3	Human-Agent-Society/CORAL: 0123dfb939

The historical Native activation profile uses Pi v0.85.1 at `d981de1229`; the later Pi role pilots instead pin `914cf1472e`, while the DSH pilots pin `99f6f02fec`. These software identities are distinct from the method-code pins above. Codex's individual run records retain its source and binary bindings; a successful strict checker does not imply that its raw binary archive was exported.

B Additional Historical Results and Interpretation

Table 8 preserves all five historical offline Formula conditions: standalone ACE and the four arms of the Combee study. All test denominators are 200. Separate runs have separate H0 measurements. Parallel arms use eight workers; the fixed batch parameter is 50. The dynamic offline trajectory mostly uses batch size one, and is not evidence of an effective large-batch speedup. The naive condition is the accepted fresh-H0 v7 replacement; earlier failures are retained in the historical archive rather than mixed into its result.

Table 8: Historical offline Formula results, seed 0. Selected-candidate private counts are shown, with an explicit selected-equals-last indicator. Canonical cost and continuous elapsed-time gaps are retained in the archived records; API upper estimates are a separate field.

Condition	Initial	Selected	Selected = last	API upper (USD)
Standalone ACE	141/200	173/200	✗	5.346
Combee study: ACE	136/200	174/200	✓	5.504
Naive parallel (v7)	139/200	168/200	✓	1.498
Fixed Combee	139/200	170/200	✓	1.633
Dynamic Combee	138/200	174/200	✓	2.976

The model-cost field in Tables 5 and 8 is an accepted physical-API upper estimate, not a canonical native ledger total or invoice. Offline continuous elapsed time is unavailable. Summing controller intervals across continuations does not recover an uninterrupted duration or process downtime. Historical MCE likewise preserves unknown canonical cost and missing native receipt coverage alongside independently retained physical-call accounting. Numeric precision in the tables is display rounding; the source JSON retains the original numbers.

C Support and Evidence Boundaries

Table 9 uses checkmarks and crosses for specific declared common-role capabilities. A cross identifies an unsupported role in this interface, not the inability of a project to implement it in the future. These are Proteus integration declarations, not a feature ranking of competing systems. They must be read

together with the live evidence matrix in Table 6. A declared role can lack a qualified benchmark, and a Native legacy adapter does not automatically provide a common-role integration.

Table 9: Declared integration roles in Proteus. ✓ = supported declaration, subject to its documented protocol; ✗ = not supplied by this common-role integration. This table is capability coverage, not empirical acceptance. Source candidates require validation and a compatible subject representation.

Integration	Ordinary actor	Candidate subject	Coding workspace	Common Native	Legacy Native
Minimal mock	✓	✗	✗	✗	✓
LLM HTTP	✓	✓	✗	✓	✓
Pi	✓	✓	✓	✓	✓
DSH	✓	✓	✓	✓	✓
Codex	✓	✓	✓	✓	✓
Aki (legacy only)	✗	✗	✗	✗	✓

D Reproducibility and Evidence Ledger

The report source snapshot is `b828a85a1fcfebedc738ba02bee19d0314b0ea04`, with production tree `0e73363387123ad067df3cfeff684af7d913fed2`. This fixed snapshot includes the companion use/contribution skills. Historical experiments and engineering results retain the source of the run that produced them; they are not relabeled as experiments on this documentation snapshot. No claim is made that its PR checks or any future release-tag checks passed.

Table 10: Evidence ledger. Records are linked at the report's documentation snapshot. Historical sources are reconstructed from the frozen archive, not replaced by the current checkout.

Evidence	Source / retained record	Supports; does not establish
Historical profiles	Frozen profile index, result aggregates, source reconstruction patches	Adapted task results and actual setup; no current-source or paper-matched rerun.
Harness-role pilots	Initial four, additional Pi, DSH/Codex eight	Sixteen minimum-cycle cells; no all-role cross-product or same-SHA gate.
Common Combee trace	Frozen-trace runner and readback	Production composition mechanics with zero provider calls; no model latency/quality effect.
Directed CORAL	Run 41394999162, source `cebc1a9`	Three-request delivery, intervention, controller recovery; no worker-crash or cooperation gain.
Complete smoke	Run 41398194611, source `1adaa8f`	Six required preparation jobs in one run; no complete raw Codex archive or future-tag acceptance.
Exports and API	Output disposition, API/schema inventory	Saved-record coverage and named public surface; no performance or researcher-productivity result.

The public source bundle contains the manuscript, editable figures, generated tables, 73 cell-level source mappings, and source-pinned URLs and SHA-256 hashes. Its evidence checker reads the recorded repository snapshot to verify numeric values; building the PDF does not require model calls or experiment execution. Table 1 has a separate manifest of source hashes and coverage definitions. Editorial notes and superseded manuscripts are retained in the authors' internal archive, not in this public bundle. This writing and readback workflow does not claim a new independent audit of every underlying raw model event. Earlier LENS/Terminal-Bench results are excluded from this report's empirical tables.

New-run workflow. A new experiment must choose an actual recipe, compatible roles, task split, evidence policy, and supported resource bounds. Validate the recipe, lock all declared sources and

plugins, then run in a fresh output directory. Resume an existing experiment only under its original identities; otherwise record a new condition. Generic builders retain `model_results_pending` until the new study has its own result evidence. Publication additionally requires the repository's exact-tag gates; accepted preparation runs are not substitutes.

Research evidence still needed. Open research questions are listed in Section 6. Matched reproduction, causal performance, complete billing, and long-horizon recursive returns remain unestablished.

E Coverage Definitions and Native Source Audit

Table 1 uses a source audit dated November 2, 2026. Penguin Harness is pinned to `7406d13e2f4b3ba0fe495feed32c172f3484be3b`, whose core package declares version 0.2.18. Proteus uses the report snapshot in Appendix D. The source bundle includes file hashes, cell definitions, and links. No competing framework was executed and no task scores were compared in this audit.

An infrastructure check means that the inspected source or documentation supplies the named mechanism; it does not establish equal enforcement, reliability, or coverage. A private-evaluation check denotes an information protocol, not a security guarantee. A method cross means absence from the inspected built-in plugin library. A question mark means that the precise common service was not established, not absence of all related workflows, logging, budget limits, or session recovery. Both systems are extensible; the table's five-method subset is not an overall breadth ranking.

Table sources and scope. The comparison uses the pinned architecture and self-improvement guides, built-in toolkit library, and trace-resume implementation (PrismShadow Team, 2026). The source manifest preserves the inspected files and each cell's definition. Proteus's corresponding stage, workflow, evidence, and operation contracts are described in Section 3. A table check establishes a declared mechanism, not a stronger isolation guarantee or a controlled quality result.

Native algorithm sources. The reference protocol is defined in `episode_protocol.py`; goal/evidence injection is implemented in `episode.py`. The common transition and its constraints are in `native_evolution.py`, with active/pending workspace execution in `harness_evolution.py`. The Native shared-evaluation fixture provides a model-free invalid-edit, repair, and private-grading example. It is mechanism evidence, not learned-quality evidence. The common profile's observed-evaluation requirement, single episode stage, and last-valid output must remain distinct from legacy Native's broader configuration options.

Algorithm versus system contribution. Native's research proposition concerns goal interpretation, evidence use, self-authored tests, and viability-gated continuation by the evolving harness. The framework makes these choices inspectable and supplies other methods for comparison. A reference method can in principle be integrated into another extensible toolkit; its scientific value therefore rests on a precise protocol and controlled evidence, not exclusivity to one software package. The four phase names and source activation plumbing alone do not establish algorithmic novelty.

Comparison still needed. Use identical starting artifacts, tasks, models, and budgets to vary broad-goal versus score-target wording, last-valid versus score-based selection, and self-evidence availability. Preserve the independent hidden evaluator and report regressions as well as improvements. Integration-effort and post-dispatch interruption studies would separately test the framework's composition and recovery claims. None of these proposed comparisons is a result of the present report.